学习情境 L05：数据基建五件套部署（6 学时）

一、学习情境基本信息

项目	内容
学习领域	LF2 平台基础设施搭建与运维
讲次	第 05 讲（第 5 周）
学时	6 学时（校内理实一体 4 学时 + 企业站实践 2 学时）
布鲁姆主层级	**应用**（记忆、理解、评价渗透）
双元场地	校内：理实一体机房（演示 + 仿真）；企业站：DAP-v2 两节点 K8S 集群（10.167.2.175 / 10.167.2.176），课程命名空间 `dap-course-g{N}`
前置	L04 已完成课程 ns 创建、kubeconfig 与 dap-v2 仓库课程分支 `course/g{N}` 就绪
企业站 PR 任务	**PR-05**：向 TiDB 目标数仓灌入零售样例数据并验证 105.9 万行可查（附验收 SQL 与结果截图）
课后费曼任务	学生讲堂②预演：任选五件套之一，讲清"它解决什么问题、去掉它会坏什么"

二、任务描述（客户委托单）

委托单编号：DAP-DEPLOY-05

某零售集团 DAP-v2 数据中台即将在两节点 K8S 集群投产。运维组委托你们小组：以 GitOps 声明式方式部署 TiDB / CRDB / Milvus / Pulsar / Redis 数据基建五件套，逐项完成健康验证，并向 TiDB 灌入零售数仓样例数据（105.9 万订单）。交付一份含 `make diff` 漂移检测证据的部署验收单，未通过项必须如实标注，不得隐瞒。

三、学习目标

1. 【记忆】说出五件套各自的版本、端口与角色定位：TiDB v8.5.8:4000（HTAP 目标数仓）、CRDB（平台库，写 26257 / 读 26267）、Milvus v2.6.6:19530（`dap_knowledge` + `dap_semantic_cache`）、Pulsar 4.0.5:6650（`jobs` 6 分区 + 应用侧 DLQ）、Redis（缓存与分布式限流）。
2. 【理解】解释每件套的选型理由与"去掉它会坏什么"，以及 GitOps"仓库为唯一事实源、先 diff 后 apply"的工作原理。
3. 【应用】在课程命名空间用 `make diff` / `make apply` 完成五件套部署，并用真实命令逐项验证健康状态。
4. 【应用】完成 TiDB 灌数前连通性验证与专库账号 `dap_app` 权限核验，为 L05 企业站灌数做准备。

5. 【评价】对照《部署验收单》逐项判定部署是否达标，对失败/未验证项如实标注 DEFERRED 及原因（HSA 诚实文化）。

四、教学重点与难点

教学重点

1. 五件套分工与"为什么选它"：TiDB HTAP 一份数据同时扛交易与分析、CRDB 强一致 + 读写分离拓扑、Milvus 高维向量近邻检索、Pulsar 持久化队列 + 死信重投、Redis 微秒级热路径缓存与限流。
2. GitOps 部署闭环：`git 仓库 → make diff（dry-run 漂移预览）→ make apply → make diff 复核归零`。

教学难点

1. 五件套功能边界易混淆（学生常问："Milvus 和 Redis 都是'缓存'？"、"Pulsar 和 Redis 的 List 都是'队列'？"）。

突破策略：用"去掉它会坏什么"反事实提问 + 对比矩阵（一致性 / 持久化 / 查询模型 / 延迟量级四列），让每组现场回答两个反事实问题后再填表。

2. 漂移（drift）概念的抽象性：live 集群状态与 git 声明的差异学生难以感知。

突破策略：实施第 3 子步骤中**亲手制造一次漂移**（手动 scale），亲眼看到 `make diff` 输出差异并修复归零，把抽象概念变成一次可复现事件。

五、教学准备

类别	内容
环境	两节点 K8S 集群（10.167.2.175/.176）可达；kubeconfig context `dap-course`；课程 ns `dap-course-g{N}`（42 个共享 ns 之一，资源配额已由运维预设）；dap-v2 仓库课程分支 `course/g{N}`
资料	`deploy/Makefile`、`deploy/k8s/base/`、`deploy/k8s/overlays/course/`；样例数据包 `data/retail_warehouse.sql.gz`（零售 105.9 万订单，ODS/DWD/DIM/DWS/ADS 全分层）；《部署验收单》模板 `docs/07-项目资源/L05-验收单.md`；五件套官方文档速查卡
工具	kubectl ≥1.28、mysql 客户端、git、python3.11 + `requirements.txt`（注意：`numpy>=1.26,<2.0` 钉版本——numpy 2.x 在 X86_V2 CPU 上触发非法指令；`pymilvus` 主键必须 int64）
分组	2 人结对（Driver/Navigator 每 20′ 轮换），全班按组号使用各自 ns 与分支

六、教学过程设计（六步法：资讯 30′/计划 30′/决策 20′/实施 90′/检查 40′/评价+学生讲堂 30′）

6.1 资讯（30′）

【教师活动】

1. 发放委托单 DAP-DEPLOY-05，用 5 分钟讲清交付物：五件套全绿 + 漂移归零 + 验收单。
2. 板书 DAP-v2 数据面拓扑：`dap-gateway(FastAPI+Strawberry GraphQL)` → 平台库 CRDB（写 `cockroachdb-a:26257` / 读 `cockroachdb-b:26267`）｜洞察 Agent → 目标数仓 TiDB `:4000`（专库账号 `dap_app`）+ Milvus `:19530`｜任务面 Pulsar `:6650`（`persistent://dap/prod/jobs`，6 分区）｜热路径 Redis。
3. 带领填写五件套选型对照矩阵（重点：每行"为什么选它"必须能说出替代品为什么不行）：

组件	版本:端口	解决的核心问题	为什么选它（对比项）	去掉它会坏什么
TiDB	v8.5.8 :4000	数仓 OLAP 分析 + HTAP	TiKV 行存 + TiFlash 列存同一份数据，免"再导一份到 ClickHouse"；国产自主可控（课程思政点）	NL2SQL 沙箱执行无处跑，105.9 万订单分析退回手工导表
CRDB	写 26257 / 读 26267	平台元数据、用户、审计的强一致存储	PostgreSQL 协议兼容（SQL 零改造）、raft 强一致、天然支持双集群读写分离拓扑	网关无平台库；读写分离（L06）失去载体
Milvus	v2.6.6 :19530	向量近邻检索	十亿级向量 ANN 索引 + 标量过滤；`dap_knowledge`（知识）与 `dap_semantic_cache`（cosine≥0.92 语义缓存）两个 collection	RAG 检索（L08）与语义缓存全部失效，问句必须每次裸调 LLM
Pulsar	4.0.5 :6650	异步任务持久化队列	BookKeeper 分层持久化、消息可重放、6 分区并行消费；应用侧 DLQ 承接 redelivery≥3 的毒丸消息（DLP 铁律）	长任务丢失、失败任务无限重试堵死队列

Redis	7.x	热点缓存 + 分布式限流	微秒级；令牌桶 Lua 脚本原子限流	每问句全量打 CRDB/TiDB，高峰限流失效雪崩

4. 穿插两个真实"依赖漂移"教训（10′）：① `numpy 2.x` 在 X86_V2 CPU 上直接 `Illegal instruction`——所以 `requirements.txt` 钉死 `numpy>=1.26,<2.0`；② `pymilvus` 插入主键必须 int64，字符串 id 报 schema 不匹配。结论：**部署清单 = 镜像版本 + 依赖版本双钉**。

【学生活动】 接收委托单；跟随拓扑图在自己纸上复画数据流；结对完成对照矩阵"去掉它会坏什么"两问；记录两个依赖坑到任务单"风险栏"。

【教学方法】 客户委托单导入 + 讲授 + 反事实提问（对比分析）。

【设计意图】 用"委托单 + 反事实"把记忆层信息（版本/端口/角色）转成理解层判断（选型权衡）；依赖坑案例为实施第 4 子步骤的版本核验埋伏笔。

6.2 计划（30′）

【教师活动】 投屏 `deploy/` 目录结构与 `Makefile` 关键目标，示范如何读部署计划：

```
deploy/
├── Makefile                  # diff / apply / ps / destroy 目标
└── k8s/
    ├── base/                 # 五件套 + dap-gateway 的 Kustomize base
    │   ├── tidb-cluster.yaml      cockroachdb-a.yaml  cockroachdb-b.yaml
    │   ├── milvus.yaml            pulsar.yaml          redis.yaml
    └── overlays/course/      # 课程 ns 补丁：资源配额收敛、NodePort、课程镜像 tag
# deploy/Makefile（节选）
NS ?= dap-course-g1
diff:   ## 漂移预览：git 声明 vs 集群 live（不改动任何资源）
        kubectl diff -k k8s/overlays/course -n $(NS)
apply:  ## 声明式应用（git 为唯一事实源）
        kubectl apply -k k8s/overlays/course -n $(NS)
ps:
        kubectl -n $(NS) get pods -o wide
```

【学生活动】 结对制定《部署计划表》：① 部署顺序及理由（提示：存储类先行，网关最后）；② 每件套的验证命令清单（每人至少写 2 条）；③ 时间盒分配（90′ 实施）；④ 风险与回滚方式（`kubectl delete -k` / `make apply` 重推）。

【教学方法】 引导文法（Leittext）：计划表只给栏目标题，内容由学生从 Makefile 与 yaml 中自行挖掘。

【设计意图】 计划环节强迫学生先读声明文件再动手，避免"上来就 apply"的冲动；验证命令清单直接供 6.4 使用，形成闭环。

6.3 决策（20′）

【教师活动】 逐组快速过计划表，只把关两件事：① 是否"**先 diff 后 apply**"（diff 非零 = 有漂移或首次部署，必须能解释每一行差异）；② 部署顺序是否考虑依赖（Redis/CRDB 先行，依赖它们的 dap-gateway 最后）。宣布纪律：**禁止 `kubectl edit/scale` 直接改 live**（DGO 铁律），一切变更改 overlay yaml 后经 `make apply`。

【学生活动】 汇报计划要点（每组 90 秒）；根据教师反馈修订计划表并签字确认。

【教学方法】 决策会（小组汇报 + 教师否决权）。

【设计意图】 双元制"决策"步骤训练工程判断：顺序与纪律比速度重要；把 GitOps 铁律变成课堂硬约束。

6.4 实施（90’）

环境变量示例：`export NS=dap-course-g1`、`export CTX=dap-course`；以下命令均在 `dap-v2/` 仓库根目录执行。输出为示例，以实际为准，证据需截图入验收单。

子步骤 1（25’）：GitOps apply 部署五件套

```
git checkout course/g1 && cd deploy
make diff NS=$NS        # 首次部署：全部资源显示 "to be created"，确认无意外删除
make apply NS=$NS
kubectl --context $CTX -n $NS get pods -o wide   # 等待 Running/Ready 1/1
```

预期（示例）：

```
NAME                          READY   STATUS    IP            NODE
cockroachdb-a-0               1/1     Running   10.244.1.11   node-176
cockroachdb-b-0               1/1     Running   10.244.2.11   node-175
tidb-0                        2/2     Running   10.244.1.12   node-176
milvus-standalone-0           1/1     Running   10.244.2.12   node-175
pulsar-broker-0               1/1     Running   10.244.1.13   node-176
redis-master-0                1/1     Running   10.244.2.13   node-175
```

*共享集群注意：42 个 ns 共享两节点，若出现 Pending，用 `kubectl -n $NS describe pod <pod>` 查看 ResourceQuota，**不要**去删别的 ns 的资源，向教师报告并等待资源窗口。*

子步骤 2（35’）：逐件健康验证（每件一条铁证）

```
# ① CRDB 双库（写/读各验一次，L06 读写分离的地基）
kubectl -n $NS exec cockroachdb-a-0 -- ./cockroach sql --insecure -e "SELECT 1;"
kubectl -n $NS exec cockroachdb-b-0 -- ./cockroach sql --insecure -e "SELECT 1;"

# ② TiDB：连通性 + 版本 + 专库账号权限（只读核验）
mysql -h 10.167.2.175 -P 4000 -u dap_app -p -e "SELECT VERSION();"
# 预期 8.5.8；再验只读（预期报权限错 = 权限收敛正确）：
mysql -h 10.167.2.175 -P 4000 -u dap_app -p -e "CREATE TABLE t(x int);"
# ERROR 1044 (42000): Access denied —— 这是"期望中的报错"，截图入验收单

# ③ Milvus：两个 collection 必须在列表中
python - <<'PY'
from pymilvus import MilvusClient
c = MilvusClient(uri="http://10.167.2.175:19530")
print(c.list_collections())          # ['dap_knowledge', 'dap_semantic_cache']
print(c.get_collection_stats("dap_knowledge"))   # {'row_count': ...} 示例
PY

# ④ Pulsar：jobs topic 6 分区必须存在
kubectl -n $NS exec deploy/pulsar-broker -- \
  bin/pulsar-admin topics list dap/prod | grep jobs
kubectl -n $NS exec deploy/pulsar-broker -- \
  bin/pulsar-admin topics partitioned-lookup persistent://dap/prod/jobs
# 预期含 "partitions:6"

# ⑤ Redis：PING + 限流脚本存在性
kubectl -n $NS run redis-probe --rm -it --image=redis:7-alpine --restart=Never -- \
  redis-cli -h redis-master ping    # PONG
```

子步骤 3（20′）：漂移制造与修复实验（本讲核心体验）

```
# ① 制造漂移：手动把 Redis 副本改成 1（违反 DGO 的教学性破坏，仅限课程 ns）
kubectl -n $NS scale deploy/redis-master --replicas=1
# ② 漂移检测：make diff 必须红（退出码非 0），指出 replicas: 2 -> 1
make diff NS=$NS || echo "DRIFT DETECTED (exit=$?)"
# ③ 以 git 为源修复：
make apply NS=$NS && kubectl -n $NS get deploy redis-master   # 回到 2
# ④ 复核归零：
make diff NS=$NS && echo "CLEAN"
```

子步骤 4（10′）：依赖钉版本核验

```
grep -n "numpy" requirements.txt        # 必须出现 numpy>=1.26,<2.0
python -c "import numpy; print(numpy.__version__)"   # 1.26.x，确认非 2.x
```

【教师活动】 巡回指导；对 Pending/CrashLoop 用 5 分钟"现场诊断术"集中讲解（`describe` → Events → 常见三类：配额、镜像拉取、探针失败）；收取每组漂移实验的前后 diff 截图。

【学生活动】 结对按子步骤执行，Driver 操作 / Navigator 对照计划表逐项勾选并记录偏差；所有验证命令输出截图归档到 `docs/07-项目资源/g{N}/L05-evidence/`。

【教学方法】 任务驱动 + 结对编程 + 现场诊断示范。

【设计意图】 实施按"部署 → 验证 → 破坏 → 修复"递进：验证培养"无证据不声称完成"，漂移实验把 DGO 铁律从条文变成肌肉记忆；子步骤 2 的 5 条命令即企业站灌数的排障工具箱。

6.5 检查（40′）

【教师活动】 发布《部署验收单》检查口径：五件套健康证据（5 项）+ diff 归零证据（1 项）+ 漂移实验证据（1 项）+ 依赖核验（1 项）= 8 项。组织**组间交叉验收**：A 组用 B 组的命令清单逐条复跑（只读命令），验证可复现性。

【学生活动】 完成自检后交换验收单；对复跑失败项在验收单"复核人"栏如实记录（不得替对方掩盖）；全班汇总共性问题 3 条贴到看板。

【教学方法】 检查表法 + 同伴互评（交叉复跑 = 最朴素的契约测试）。

【设计意图】 "可复现"是工程证据的底线；交叉验收同时暴露文档不完整问题（截图 ≠ 可复跑），呼应 HSA 诚实文化。

6.6 评价 + 学生讲堂（30′）

【教师活动】 按第八节评分表现场评分（抽查 2 组答辩 3 分钟：任指一件套问"为什么选它"）；主持学生讲堂②：随机抽 3 人，每人 3 分钟讲"一个中间件解决什么问题、去掉会坏什么"，不能用 PPT，只许白板；宣布课后费曼任务与下周 L06 预习（读写分离代码 `services/gateway/app/db/dual_pool.py` 通读一遍）。

【学生活动】 被抽讲学生白板讲解；听众用互评量表（讲明白/有例子/答上反问 三项各 ★1-3）；未抽到的学生完成自评三问（见第九节）。

【教学方法】 费曼学生讲堂 + 同伴互评量表。

【设计意图】 3 分钟无 PPT 讲解逼学生用自己的话重构知识（费曼法核心）；互评量表给出可操作的标准而非印象分。

七、企业站实践任务卡（2 学时）

任务卡 PR-05：TiDB 灌数验证（105.9 万行）

项目	内容
任务目标	向 TiDB 目标数仓灌入零售样例数据，验证 105.9 万订单全量可查、抽样业务问题可答
前置条件	L05 校内部署验收 8 项中 ≥6 项通过；dap_app 账号可连 10.167.2.175:4000
结对方式	2 人结对，Driver/Navigator 30′ 轮换；在企业导师见证下操作课程分支

步骤：

```
# 1. 灌数（脚本内部按 ODS→DWD/DIM→DWS→ADS 顺序执行，幂等可重跑）
mysql -h 10.167.2.175 -P 4000 -u root -p < scripts/seed/create_retail_schema.sql
zcat data/retail_warehouse.sql.gz | \
  mysql -h 10.167.2.175 -P 4000 -u root -p --local-infile=1 retail_dw

# 2. 行数铁证（预期 1,059,3xx，以实际灌数为准，截图入 PR）
mysql -h 10.167.2.175 -P 4000 -u dap_app -p -e \
 "SELECT COUNT(*) FROM retail_dw.ods_orders;"        # ≈ 1,059,000+
mysql -h ... -e "SELECT COUNT(*) FROM retail_dw.dwd_order_items;"
mysql -h ... -e "SELECT COUNT(*) FROM retail_dw.dim_products;"

# 3. 三条业务抽验 SQL（必须用 dap_app 只读账号执行）
# Q1 品类销售 Top3：
SELECT p.category, SUM(i.pay_amount) amt FROM dwd_order_items i
  JOIN dim_products p ON i.product_id=p.product_id
  GROUP BY p.category ORDER BY amt DESC LIMIT 3;
# Q2 月度订单量：SELECT DATE_FORMAT(order_ts,'%Y-%m') m, COUNT(*) FROM dwd_orders GROUP BY m;
# Q3 客户复购率粗算：按 customer_id 分组统计 ≥2 单占比

# 4. 权限终验：确认 dap_app 无法 INSERT/DDL（预期 Access denied 截图）
```

交付物（PR 内容）：

1. 分支 course/g{N} → main 的 PR，含：验收单 docs/07-项目资源/g{N}/L05-验收单.md、证据截图目录、docs/07-项目资源/g{N}/L05-灌数报告.md（行数、三条抽验 SQL 及结果、异常记录）。
2. 灌数报告中必须含"**未验证/未完成清单**"栏（如某 DWS 表行数异常未排查，写明现象与下一步）。

评审标准（企业导师三查）：

- 查证据：行数截图与 SQL 结果一一对应，无"声称完成"无证据项；
- 查规范：命令可复现（他人按 PR 可重跑）、只读账号用对、无 root 跑业务查询；
- 查诚实：失败与未验证项如实列出；虚报一项即打回重做。

双导师签栏	意见	签名/日期
校内导师		

企业导师		

八、评价设计（任务单评分表）

维度	权重	90-100 分标准	75-89 分标准	60-74 分标准	<60 分标准
选型理解（资讯）	15%	对照矩阵五项全对，能答任意反事实追问	四项对，反事实答出大意	三项对，靠提示	两项以下或照抄
部署规范（实施）	25%	先 diff 后 apply 全程零手动改 live；Pending 自诊断	个别步骤顺序颠倒但自纠	需教师提示完成	直接 apply 或改 live
验证证据（实施）	25%	五件套 5 条命令输出全截图且可复跑	4 条	3 条	≤2 条
漂移实验	15%	制造→检测→修复→归零四步证据齐全	缺归零复核	只完成制造+检测	未做
交叉验收与协作	10%	复跑他组全部通过并给出改进建议	复跑通过	部分通过	未参与
学生讲堂	10%	白板讲清问题+反例，答上追问	讲清问题	复述概念	讲不出

总评 = Σ(维度分 × 权重)；企业站 PR-05 另按 15% 计入课程总评（见课程标准 §五）。**红线**：验收单虚报（写"通过"但无证据）该维度按 0 分计并要求重做。

九、课后费曼任务

项目	内容
讲给谁	一位非计算机专业的同学/家人（完全不懂数据库的人）
讲什么	任选五件套之一（建议选你今天踩坑最多的那个），用 3 分钟讲清三件事：① 它解决什么问题（用一个生活化类比）；② 去掉它整个平台会坏什么；③ 它和最容易被混淆的那个组件（如 Milvus vs Redis）差在哪。录音或文字记录对方听懂没有
自评三问	① 我有没有用到"向量""持久化"这类对方听不懂的词而没解释？② 对方能不能用自己的话复述"去掉它会坏什么"？③ 如果对方问"为什么不直接用 MySQL 存所有东西"，我能答上来吗？

十、教学反思栏

反思维度	记录（课后填写）
目标达成情况（五件套验证通过率）	
学生卡点（Pending/依赖坑/漂移理解）	
时间分配偏差（实施 90′ 是否够用）	
交叉验收暴露的共性问题	
下一讲（L06）需要的补课点	